CMPE 273 – Distributed Systems Week 2: Enterprise Software Architecture

From Monoliths to Agentic Distributed Systems

Rakesh Ranjan

Why Architecture Matters

Monolith

One Big Kitchen

SOA

Coordinated Food Court

Microservices

Independent Food Trucks

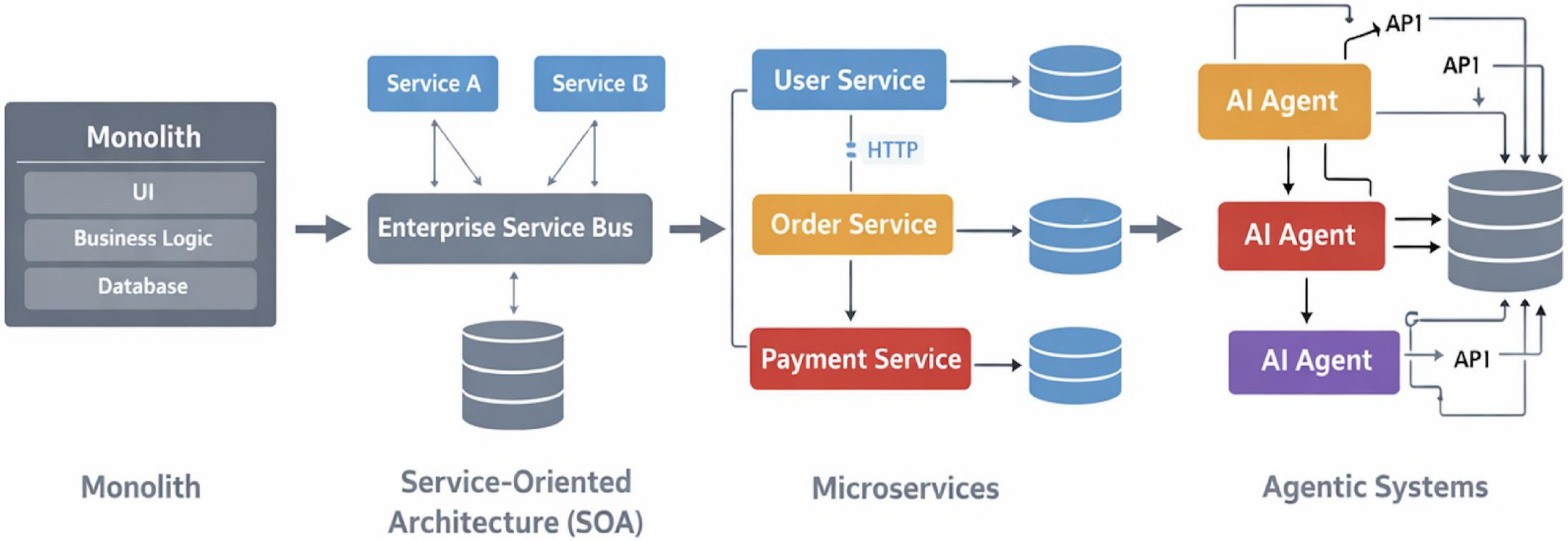
Agentic System

Autonomous Pop-Ups

- Architecture encodes assumptions about failure
- Scale is about teams, velocity, and change
- Distributed systems exist because organizations scale
- Most outages are architectural, not code bugs

From One Big Kitchen to Distributed AI Chefs

Evolution of Enterprise Architectures



Monolithic Architecture

Single deployable unit

Real-life system: Single-kitchen restaurant (Monolith)
Single deployable unit

- One kitchen, one staff, one menu, one opening time.
- If the restaurant opens, *everything* opens together.
  If the kitchen is closed for renovation, the entire restaurant shuts down.

Mapping to software:
You deploy the whole application as one artifact. No partial rollout.

Shared memory and state

Shared memory and state
One whiteboard for orders. One fridge. One pantry. One cash register.

- All chefs, servers, and the cashier read and write to the same shared resources.

Failure mode:
If the order board is erased or the fridge breaks, *everyone* is affected.
Mapping to software:
Shared in-process memory, shared database connections, shared state.

Monolithic Architecture (2)

Simple to build, hard to scale

Easy to start with 3 chefs and 10 tables.
Impossible when 100 customers arrive at once.
You cannot scale just the grill station. You must:

- Hire more staff everywhere
- Expand the entire kitchen
- Retrain everyone

Mapping to software:
You scale the whole app even if only one part is under pressure.

Failure is total and obvious

Gas line fails → kitchen stops → restaurant closes → customers leave.
No partial degradation. No graceful fallback.
Mapping to software:
One crash brings everything down.
Outage is loud, visible, and complete.

Service-Oriented Architecture (SOA)

Real-life system: Food court with a central coordinator (SOA)
Think **mall food court** with shared infrastructure and strict rules.

Services with centralized governance

Each stall is a service:

- Pizza stall, Sushi stall, Dessert stall

But the mall management controls:

- Operating hours, Menu formats, Payment system, Pricing rules, Health inspections
- No stall is truly independent.

Mapping to software

- Services exist, but governance is centralized.
- Teams must comply with shared standards and approval processes.

Often built around ESBs

All orders must go through a **central ordering counter**. Customer does not talk directly to stalls. The counter:

- Takes orders, Translates requests, Routes them to stalls, Handles payments

Failure mode

If the counter slows down, *every stall slows down*.

Mapping to software

Enterprise Service Bus becomes:

- Routing hub, Transformation layer, Policy enforcement point

Service-Oriented Architecture (SOA) (2)

Improves reuse but increases coordination cost

Shared services:

- One payment system, One loyalty program, One delivery partner

Great reuse. But:

- Changing payment flow requires approval
- Adding a new menu item requires coordination
- One team's change affects all stalls

Mapping to software

- Reuse increases, Delivery speed decreases.

Becomes rigid at scale

As more stalls join:

- The order counter becomes overloaded
- Rule changes take weeks
- Innovation slows because everything must be standardized

Eventually:

- New stalls stop opening
- Existing stalls work around rules informally

Mapping to software

SOA becomes:

- Process heavy, Slow to evolve, Resistant to change

Microservices Architecture

Example: A city with **independent food trucks** operating across neighborhoods.

Independently deployable services

Each food truck:

- Owns its menu
- Chooses operating hours
- Decides pricing
- Can shut down or restart independently

One truck can fail without closing the city.

Mapping to software

Each service:

- Built and deployed independently
- Owned by one team
- Has its own runtime and data

API-first communication

Food trucks communicate via:

- Online ordering platforms
- Delivery apps
- Standard pickup instructions
- No central counter. Just contracts.

Mapping to software

REST, gRPC, events.

Clear contracts replace shared memory.

Microservices Architecture (2)

Team autonomy

When demand spikes:

- Add more trucks
- Move trucks to busy neighborhoods
- Scale only what's popular

Mapping to software

Scale hot services without touching others.

Distributed failure and debugging complexity

Customer orders from:

- Taco truck
- Dessert truck
- Drink truck

One truck running late affects the experience, but no single owner sees the whole picture.

Mapping to software

Failures are:

- Partial
- Hard to debug
- Spread across services

Agentic and Autonomous Services

Example: Autonomous pop-up kitchens

AI-driven pop-up kitchens that:

- Sense demand in real time
- Decide where to operate
- Coordinate with each other
- Learn from past outcomes

No human dispatcher.

Services augmented with reasoning

A pop-up kitchen decides:

- Where to open tonight
- What menu to offer
- How much inventory to prep

Based on:

- Weather, Past sales, Local events, Traffic data

Mapping to software

Services don't just execute, They **decide**.

Control loops instead of static workflows

If demand drops:

- Kitchen relocates
- Menu changes
- Prep volume adjusts

Continuously.

Mapping to software

Feedback loops replace fixed flows.

Observation → decision → action.

Agentic and Autonomous Services (2)

Example: Autonomous pop-up kitchens

Human + agent collaboration

Humans:
- Set boundaries
- Review outcomes
- Intervene in edge cases

Agents:
- Handle the routine decisions
- Optimize continuously

Mapping to software
Humans supervise.
Agents operate.

Architecture must handle non-determinism

Two pop-ups make different choices with the same data.
Both are “reasonable.”
Mapping to software
Failures are:
- Subtle
- Probabilistic
- Harder to reproduce

Observability and guardrails become essential.

Takeaway- Agentic systems don’t remove complexity. They move it into decision-making.

1. **Monolith** – one kitchen
2. **SOA** – food court with rules
3. **Microservices** – food trucks
4. **Agentic systems** – autonomous pop-ups

Synthesis Question:
Compare the dominant failure modes of Monoliths, SOA, Microservices, and Agentic Systems using a different real-world analogy.

Communication Models

- Synchronous: REST, gRPC
- Asynchronous: Queues, Events
- Streaming: Kafka-style pipelines

Synchronous request response

What is it?

- Caller waits for callee to respond
- Typically HTTP REST or gRPC

Enterprise example

- Checkout Service calls Payments Service: authorize(card, amount)
- If Payments is slow or down, checkout stalls

Simple example

`OrderService` calls `InventoryService` to reserve items before confirming order

Failure modes to highlight

- Timeout cascades
- Retry storms
- Thread pool exhaustion
- Latency amplification across dependency chains (A calls B calls C)

Asynchronous messaging (queues, pub sub)

What is it?

- Producer sends a message and moves on
- Consumer processes later
- Usually via message broker like RabbitMQ, SQS, Kafka (as pub sub)

Enterprise example

User signs up → emit **UserCreated**

Downstream services react:

- Email Service sends welcome email
- CRM Service creates lead
- Analytics Service logs event

Simple example

- `UserService` publishes `UserCreated`
- `NotificationService` consumes and sends email

Failure modes

- Duplicate delivery (at-least-once semantics)
- Ordering issues
- Poison messages
- Consumer lag and backlog growth
- Exactly-once is expensive or unrealistic in many systems

Streaming event pipelines

What is it?

- Continuous stream of ordered events
- Consumers can replay history
- Think Kafka or Pulsar
- High volume
- Durable event history
- Many consumers, evolving over time

Simple example

- ClickstreamProducer publishes events continuously
- AnalyticsConsumer computes rolling metrics (per minute counts, top pages)

Failure modes

- Reprocessing bugs and double counting
- Schema evolution breaking consumers
- Hot partitions
- Lag, backpressure, and stateful consumer failures

Enterprise example

Payment events stream:

- Fraud detection consumes in near-real time
- Ledger service builds immutable accounting view
- Support analytics computes incident correlation

Architecture is Operations

- Observability is not optional
  - Logs, metrics, and traces are architectural primitives
  - You cannot debug what you cannot see

In-Class Activity: Architecture Tradeoffs

Design a campus-wide food ordering system with:

- Order placement
- Inventory reservation
- Notification to user
- Metrics collection

You must evaluate **three communication models**:

1. Synchronous
2. Asynchronous messaging
3. Streaming

For each architecture style and communication model:
Identify acceptable failures
Call out one mistake you will avoid

Model	What fails first	What user sees	What keeps working
Synchronous			
Async			
Streaming			

Assignment 1: Architecture Decomposition

Goal: Implement the same “campus food ordering” workflow three ways:

- Synchronous REST
- Async messaging with RabbitMQ
- Streaming with Kafka

Each part includes: build, run, test, and failure injection. I will share a recommended repo structure and Services end-points